

Lecture 3: Scikit-learn, TensorFlow and Keras

Yaxin Bi
School of Computing
University of Ulster

1

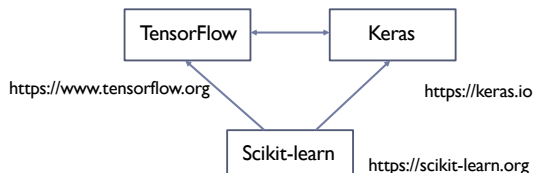
Contents

- ▶ Introduction to Deep Learning Framework
- ▶ Introduction to Scikit-Learn
- ▶ The 5-Step Deep Learning Model Life-Cycle
- ▶ Keras APIs in classification and Regression

2

Deep Learning Framework and APIs

- ▶ TensorFlow is the core open source library to help you develop and train Deep Learning models – a development framework
- ▶ Keras is an open-source deep learning library written in Python.
- ▶ Scikit-learn is simple and efficient tools for developing data analytics built on NumPy, SciPy, and matplotlib



3

TensorFlow Core API

- ▶ The TensorFlow developed by Google Brain since 2011, and offered both Python and C++ API. The Core API requires working with
 - ▶ computational graphs
 - ▶ tensors
 - ▶ operations
 - ▶ sessions
- ▶ some of which can be hard to understand for the beginners who just start to work with TensorFlow.
- ▶ There are some advantages to using the low-level TensorFlow Core API, mostly when debugging
- ▶ We can mix the high-level and low-level TensorFlow APIs as required.

4

Keras API

- ▶ TensorFlow is the deep learning library, while Keras offers a simple and powerful Python API for accessing TensorFlow.
- ▶ TensorFlow 2 provides full Keras integration, making advanced machine learning easier and more convenient than ever before.
- ▶ The API was “designed for human beings, not machines,” and “follows best practices for reducing cognitive load.”
- ▶ The Model is the core Keras data structure. There are two main types of models available in Keras:
 - ▶ **Sequential** model,
 - ▶ Model class used with the **functional** API.

5

Keras API (cont'd)

- ▶ API includes:
 - ▶ Neural layers
 - ▶ cost functions
 - ▶ optimizers
 - ▶ initialization schemes
 - ▶ activation functions and regularization schemes
- ▶ all of which are all standalone modules that you can combine to create new models.
- ▶ New modules are simple to add, as new classes and functions.
- ▶ Models are defined in Python code, not separate model configuration files.

6

Scikit-Learn

- Scikit-learn is an open source machine learning library that supports supervised and unsupervised learning.
- It also provides various tools for model fitting (training), data preprocessing, model selection and evaluation, and many other utilities.
- Here are some examples
 - Linear Models
 - Kernel ridge regression
 - Support Vector Machines
 - Stochastic Gradient Descent
 - Nearest Neighbors
 - Naive Bayes
 - Decision Trees
 - Neural networks

7

A Multi-layer Perceptron Model in Scikit-learn: Classification

- A Multilayer Perceptron model (MLP) is a standard fully connected neural network model.
- Scikit-Learn implements the MLP algorithm with a Class called **MLPClassifier** that trains using **Backpropagation**
- To invoke this class to create a MLP model, we need to initiate the class with appropriate parameters
 - MLP trains on two arrays: array **X** of size (n_samples, n_features), which holds the training samples represented as floating point feature vectors; and array **y** of size (n_samples,) which holds the target values (class labels) for the training samples:


```
from sklearn.neural_network import MLPClassifier
X = [[0., 0.], [1., 1.]]
y = [0, 1]
clf = MLPClassifier(solver='lbfgs', alpha=1e-5, hidden_layer_sizes=(5, 2), random_state=1)
clf.fit(X, y)
print(clf.predict([[2., 2.], [-1., -2.]]) → [1 0]
```

8

A Multi-layer Perceptron Model in Scikit-learn (cont'd)

- Currently, **MLPClassifier** supports only the Cross-Entropy loss function, which allows probability estimates by running the predict_proba method.
- In respect of using Backpropagation, it trains using some form of gradient descent and the gradients are calculated using Backpropagation. For classification, it minimizes the Cross-Entropy loss function, giving a vector of probability estimates $P(y|x)$ per sample x :

```
clf.predict_proba([[2., 2.], [1., 2.]])
[[2.07145538e-04 9.99792854e-01]
 [2.07145538e-04 9.99792854e-01]]
```

9

A Multi-layer Perceptron Model in Scikit-learn (cont'd)

- MLPClassifier** supports multi-class classification by applying **Softmax** as the output function.
- in which a sample can belong to more than one class. For each class, the raw output passes through the logistic function. Values larger or equal to 0.5 are rounded to 1, otherwise to 0.
- For a predicted output of a sample, the indices where the value is 1 represents the assigned classes of that sample:

```
X = [[0., 0.], [1., 1.]]
y = [[0, 1], [1, 1]]
clf = MLPClassifier(solver='lbfgs', alpha=1e-5, hidden_layer_sizes=(15), random_state=1)
clf.fit(X, y)
print(clf.predict([[1., 2.]]) → [[1 1]]
print(clf.predict([[0., 0.]]) → [[0 1]]
```

10

A Multi-layer Perceptron Model in Scikit-learn (cont'd): Regression

- Scikit-Learn implements Class **MLPRegressor** that trains using backpropagation with no activation function in the output layer, which can also be seen as using the identity function as activation function.
- As **MLPRegressor** does not use an activation functions instead uses the square error as the loss function, and the output is a set of continuous values.
- MLPRegressor also supports multi-output regression, in which a sample can have more than one target.

11

Steps for Developing a DL Model by Keras

- Define the model
- Compile the model
- Fit the model
- Evaluate the model
- Make predictions

12

Defining the Model

- ▶ First select the type of model that you need and then choose the architecture or network topology.
- ▶ From an API perspective, this mainly involves
 - ▶ defining the **layers** of the model
 - ▶ configuring each layer with a number of **nodes**
 - ▶ selecting **activation function**,
 - ▶ and connecting the layers together into a cohesive model.
- ▶ Models can be defined either with
 - ▶ Sequential API or
 - ▶ Functional API,
- ▶ The Sequential model is a linear **stack** of layers, simply uses `model.add()` to define dense layers

▶ 13

13

Compile the Model

- ▶ First select a loss function that is used to optimize algorithms
 - ▶ `'binary_crossentropy'` for binary classification.
 - ▶ `'sparse_categorical_crossentropy'` for multi-class classification.
 - ▶ `'mse'` (mean squared error) for regression.
- ▶ Select an algorithm to perform the optimization procedure,
 - ▶ typically stochastic gradient descent (SGD), or a modern variation, such as Adam.
- ▶ Select any performance metrics to keep track of the model training process.
 - ▶ Classification accuracy, F-measure
- ▶ From an API perspective, this involves calling **'compile'** function to compile the model with the chosen configuration, preparing the appropriate data structures required for the efficient use of the model defined.

▶ 14

14

Example

- ▶ **# define the model**
- ▶ `model = Sequential()`
- ▶ ...
- ▶ **# compile the model**
- ▶ `opt = SGD(learning_rate=0.01, momentum=0.9)`
- ▶ `model.compile(optimizer=opt, loss='binary_crossentropy')`
- ▶ Use SGD with the default settings:
- ▶ `model.compile(optimizer='sgd', loss='mse')`
- ▶ ...
- ▶ **# compile the model with a metrics**
- ▶ `model.compile(optimizer='sgd', loss='binary_crossentropy', metrics=['accuracy'])`

▶ 15

15

Fit the Model (Training)

- ▶ Select the training configuration
 - ▶ The number of **epochs** (loops through the training dataset) and the **batch size** (number of samples in an epoch used to estimate model error).
- ▶ Fitting applies the chosen optimization algorithm to minimize the chosen loss function and updates the model using the backpropagation of error algorithm.
- ▶ Fitting the model is the slow part of the whole process and can take seconds, hours, days, months, depending on the complexity of the model, hardware, and the size of the training dataset.
- ▶ For APIs, this calls the **'fit'** function to perform the training process. This function will block (not return) until the training process has finished.

▶ 16

16

Evaluate the Model

- ▶ Evaluating the model requires
 - ▶ choosing a holdout dataset used to evaluate the model.
 - ▶ dataset should not used in the training process so that we can get an unbiased estimate of the performance of the model when making predictions on new data.
- ▶ The speed of model evaluation is proportional to the amount of data
 - ▶ it is much faster than training as the model is not changed.
- ▶ This involves calling the API **'evaluate'** function with the holdout dataset and getting a loss and perhaps other metrics that can be reported.

▶ 17

17

Make a Prediction

- ▶ Making a prediction is the final step in the life-cycle. It is why we wanted the model in the first place.
- ▶ Prediction requires a new dataset, which has not been seen in training process
- ▶ We simply call an API **'predict'** function to make a prediction of a class label, probability, or numerical value: whatever we designed our model to predict.
- ▶ We may want to save the model and load it later to make predictions. We may also choose to fit a model on all of the available data before you start using the model.

▶ 18

18

Example

```
# fit the model
model.fit(X, y, epochs=100, batch_size=32)
model.fit(X, y, epochs=100, batch_size=32, verbose=0)
▶ Using 'verbose' to simplify the report of model performance each epoch by setting verbose=2
▶ All output can be turned off during training by setting verbose=0.
...
# evaluate the model
loss = model.evaluate(X, y, verbose=0)
...
# make a prediction
yhat = model.predict(X)
```

▶ 19

19

Sequential Model API (Simple)

- ▶ The sequential model API is the simplest and is the API developed by Keras, it is referred to as '**sequential**'
- ▶ We initiate a model by the **Sequential class** and adds layers to the model one by one in a linear manner, from input to output.
- ▶ The example below defines a Sequential MLP model that accepts eight inputs, has one hidden layer with 10 nodes and then an output layer with one node to predict a numerical value.
- ▶ Example


```
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense
model = Sequential()
model.add(Dense(10, input_shape=(8,)))
model.add(Dense(1))
```

▶ 20

20

Sequential Model API (cont'd)

- ▶ The visible layer of the network is defined by the "**input_shape**" argument on the first layer. The model expects the input for one sample to be a vector of eight numbers.
- ▶ The sequential API is easy to use because we keep calling **model.add()** until we have added all of layers required.

```
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense
model = Sequential()
model.add(Dense(100, input_shape=(8,)))
model.add(Dense(80))
model.add(Dense(30))
model.add(Dense(10))
model.add(Dense(5))
model.add(Dense(1))
```

▶ 21

21

Functional Model API (Advanced)

- ▶ The functional API is more complex but is also more flexible.
- ▶ It involves explicitly connecting the output of one layer to the input of another layer. Each connection is specified.
- ▶ First, an input layer must be defined via the **Input** class, and the shape of an input sample is specified. We must retain a reference to the input layer when defining the model.


```
# define the layers
x_in = Input(shape=(8,))
```
- ▶ Next, a fully connected layer can be connected to the input by calling the layer and passing the input layer. This will return a reference to the output connection in this new layer.

▶ 22

22

Functional Model API (cont'd)

```
x = Dense(10)(x_in)
▶ We can then connect this to an output layer in the same manner.
x_out = Dense(1)(x)
▶ Once connected, we define a Model object and specify the input and output layers. The complete example is listed below.
from tensorflow.keras import Model
from tensorflow.keras import Input
from tensorflow.keras.layers import Dense
# define the layers
x_in = Input(shape=(8,))
x = Dense(10)(x_in)
x_out = Dense(1)(x)
# define the model
model = Model(inputs=x_in, outputs=x_out)
```

▶ 23

23

Develop MLP Models

- ▶ MLP is comprised of layers of nodes where each node is connected to all outputs from the previous layer and the output of each node is connected to all inputs for nodes in the next layer.
- ▶ An MLP is defined with one or more **Dense layers**.
- ▶ This model is appropriate for tabular data that looks a table with one column for each feature and one row for data point
- ▶ There are three predictive modeling problems that can be explored with an MLP: binary classification; multiclass classification; and regression.
- ▶ Let's look at an MLP model on a real dataset for each of these cases.

▶ 24

24

MLP for Binary Classification

- ▶ Use the Ionosphere binary (two-class) classification dataset to demonstrate an MLP for binary classification.
- ▶ This task involves predicting whether a structure is in the atmosphere or not given radar returns.
 - there are 351 instances (data points) in total
 - 34 features plus one class label
 - Feature information – all 34 are continuous, as described above
 - The 35th feature is either "good" or "bad" according to the definition summarized above., which is a binary classification task.

▶ 25

25

MLP for Binary Classification (cont'd)

- ▶ The dataset can be downloaded automatically using **Pandas**, which is third party package
- ▶ We will use a **LabelEncoder** third party class to encode the string labels 'good' and 'bad' to integer values 0 and 1.
- ▶ The model will be **trained** on 67 percent of the data, and the remaining 33 percent will be used for evaluation, split using the **train_test_split()** function.
- ▶ Defining activation functions
 - ▶ ReLU with with a 'he_normal' weight initialization
 - ▶ Using the **sigmoid** activation function for binary classification
 - ▶ Using the **adam** version of stochastic gradient descent as an optimizer
 - ▶ Minimize the cross-entropy loss – 'binary_crossentropy'

▶ 26

26

Normalizing Labels

- ▶ LabelEncoder class help normalize labels, encoding target labels with value between 0 and n_classes-1
- ▶ It can also be used to transform non-numerical labels (as long as they are hashable and comparable) to numerical labels
- ▶ This normalizer should be used to encode target values, i.e. y, rather than the input X.

```
le1 = preprocessing.LabelEncoder()
le1.fit(["paris", "paris", "tokyo", "amsterdam"])

['amsterdam', 'paris', 'tokyo']
[0 2 1]
```

▶ 27

27

```
# define model
model = Sequential()
model.add(Dense(10, activation='relu', kernel_initializer='he_normal',
input_shape=(n_features,)))
model.add(Dense(8, activation='relu', kernel_initializer='he_normal'))
model.add(Dense(1, activation='sigmoid'))
# compile the model
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
# fit the model
model.fit(X_train, y_train, epochs=150, batch_size=32, verbose=0)
# evaluate the model
loss, acc = model.evaluate(X_test, y_test, verbose=0)
print("Test Accuracy: %.3f % acc)
# make a prediction
row = [1,0,0.99539,-0.05889,0.85243,0.02306,0.83398,-
0.37708,1,0.03760,0.85243,-0.17755,0.59755,-0.44945,0.60536,-0.38223,0.84356,-
0.38542,0.58212,-0.32192,0.56971,-0.29674,0.36946,-0.47357,0.56811,-
0.51171,0.41078,-0.46168,0.21266,-0.34090,0.42267,-0.54487,0.18641,-0.45300]
yhat = model.predict(row)
print("Predicted: %.3f % yhat)
```

▶ 28

28

MLP for Multiclass Classification

- ▶ Use the Iris flowers multiclass classification dataset to demonstrate an MLP for multiclass classification.
- ▶ This dataset is used to predict whether a flower is in one of types of Iris flowers
 - There are 150 instances – 50 in each of three classes
 - 4 features plus predictive label
 - Attribute Information: 1. sepal length in cm 2. sepal width in cm 3. petal length in cm 4. petal width in cm
 - Class labels: Iris Setosa; Iris Versicolour; Iris Virginica

▶ 29

29

MLP for Multiclass Classification (cont'd)

- ▶ The dataset can be downloaded automatically using **Pandas**, which is third party package
- ▶ We will use a **LabelEncoder** third party class to encode the string labels 'Iris Setosa'; 'Iris Versicolour'; 'Iris Virginica' to integer values 0, 1, 3.
- ▶ The model will be **trained** on 67 percent of the data, and the remaining 33 percent will be used for evaluation, split using the **train_test_split()** function.
- ▶ Defining activation functions
 - ▶ The model must have one node for each class in the output layer and
 - ▶ Use the **softmax** activation function
 - ▶ Minimize the cross-entropy loss – 'sparse_categorical_crossentropy'

▶ 30

30

```
# define model
model = Sequential()
model.add(Dense(10, activation='relu', kernel_initializer='he_normal',
input_shape=(n_features,)))
model.add(Dense(8, activation='relu', kernel_initializer='he_normal'))
model.add(Dense(3, activation='softmax'))
# compile the model
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy',
metrics=['accuracy'])
# fit the model
model.fit(X_train, y_train, epochs=150, batch_size=32, verbose=0)
# evaluate the model
loss, acc = model.evaluate(X_test, y_test, verbose=0)
print("Test Accuracy: %.3f" % acc)
# make a prediction
row = [5.1, 3.5, 1.4, 0.2]
yhat = model.predict([row])
print("Predicted: %s (class=%d)" % (yhat, argmax(yhat)))
```

▶ 31

31

MLP for Regression

- ▶ A regression problem only involves predicting a single numerical value
- ▶ Using Boston housing dataset to demonstrate an MLP for regression predictive modeling
- ▶ This problem involves predicting house value based on properties of the house and neighbourhood
- ▶ Note that this is a regression, not classification; therefore, we cannot calculate classification accuracy. Instead we only can calculate a **mean squared error** or other types of errors
- ▶ Boston housing dataset is following

▶ 32

32

Boston Housing dataset

Number of Instances: 506

Number of Attributes: 13 continuous attributes (including "class" attribute "MEDV").
Attribute Information:

1. CRIM per capita crime rate by town
2. ZN proportion of residential land zoned for lots over 25,000 sq.ft.
3. INDUS proportion of non-retail business acres per town
4. CHAS Charles River dummy variable (= 1 if tract bounds river; 0 otherwise)
5. NOX nitric oxides concentration (parts per 10 million)
6. RM average number of rooms per dwelling
7. AGE proportion of owner-occupied units built prior to 1940
8. DIS weighted distances to five Boston employment centres
9. RAD index of accessibility to radial highways
10. TAX full-value property-tax rate per \$10,000
11. PTRATIO pupil-teacher ratio by town
12. B $1000(Bk - 0.63)^2$ where Bk is the proportion of blacks by town
13. LSTAT % lower status of the population
14. MEDV Median value of owner-occupied homes in \$1000's

▶ 33

33

MLP for Regression (cont'd)

- ▶ The dataset can be downloaded automatically using **Pandas**, which is third party package
- ▶ There is no need to a **LabelEncoder** third party class to encode the string labels to integer values 0 and 1.
- ▶ The model will be **trained** on 67 percent of the data, and the remaining 33 percent will be used for evaluation, split using the **train_test_split()** function.
- ▶ Defining activation functions
 - ▶ The model must have one node for each class in the output layer
 - ▶ Output layer has a single node and uses the default or linear activation function (no activation function).
 - ▶ Minimize the mean squared error (mse) loss

▶ 34

34

```
# define model
model = Sequential()
model.add(Dense(10, activation='relu', kernel_initializer='he_normal',
input_shape=(n_features,)))
model.add(Dense(8, activation='relu', kernel_initializer='he_normal'))
model.add(Dense(1))
# compile the model
model.compile(optimizer='adam', loss='mse')
# fit the model
model.fit(X_train, y_train, epochs=150, batch_size=32, verbose=0)
# evaluate the model
error = model.evaluate(X_test, y_test, verbose=0)
print("MSE: %.3f, RMSE: %.3f" % (error, sqrt(error)))
# make a prediction
row = [0.00632, 18.00, 2.31, 0.0, 0.5380, 6.5750, 65.20, 4.0900, 1.296, 0.1530, 396.90, 4.98]
yhat = model.predict([row])
print("Predicted: %.3f" % yhat)
```

▶ 35

35